

# The willThrow Tax

What a test-only runtime hook does to code that throws at volume — and how to get out from under it.

Under Apple's test frameworks, every Swift throw is intercepted by a global runtime hook. For ordinary tests it is invisible. For code that throws millions of times — fuzzers, property tests, stress suites — it turns a fast loop slow and quietly leaks memory until the run runs out of RAM. This is the full trace, from the first symptom to the root cause, with reproducible numbers and fixes.

- 1 Detection: how it showed up
- 2 Root cause: a global hook on every throw
- 3 Measurement: the same loop, three places
- 4 The second problem: it leaks memory
- 5 The full trace: twenty-one findings
- 6 The multiplier: throws you never wrote
- 7 Fixes: what actually gets you out
- 8 Verdict
- 9 Reproduce it
- 10 Credits

**~36×**

slower per throw

**2.1 KB**

leaked per throw

**21**

findings

**0×**

cost in production

**September 2026**

MacBook Pro M5 · Swift 6.3.3 · macOS 26 · Xcode 26.6 · release unless noted

# Under Apple's test frameworks, every throw is intercepted

For ordinary tests the hook is invisible. For code that throws millions of times — fuzzers, property tests, stress suites — it turns a fast loop slow and quietly leaks memory until the run runs out of RAM. This is the full trace, from the first symptom to the root cause, with reproducible numbers and fixes.

## SCOPE, STATED UP FRONT

This costs nothing in shipped apps — the hook is inert outside a test process. It never affects correctness. It only bites a narrow workload: throwing at very high volume inside a test. It is a **sharp edge, not a broken framework**. The one part that is a genuine defect is the memory growth.

**~36x**

slower per throw, XCTest vs executable

**2.1 KB**

leaked per throw, until the test ends

**0x**

cost in a shipped app

## §1 · DETECTION

### How the problem showed up

It surfaced while fuzzing Kalego, an encrypted messenger, against the eight parsers that read data off the network. Three of those parsers throw on bad input — by design. So under the fuzzer, throwing was not the rare case. It was the common case, hundreds of millions of times.

The run stalled. One core pinned at 100% for over an hour while the others sat idle. No crash. No error. No leak warning. Memory looked flat. Everything «worked» — it was just impossibly slow.

```
# find the test process, then sample its stacks
pgrep -f xctest
sample <pid> 5 -f /tmp/sample.txt
```

The sampled stack, top (deepest) to bottom, named the culprit:

```
your loop
  parse(...)
    swift_willThrowTypedImpl          <- fires on every throw
    XCTSwiftErrorObservation...      <- XCTest is watching
    _swift_stdlib_bridgeErrorToNSError <- builds an NSError
    _getErrorUserInfoNSDictionary
    ...CFBasicHashFindBucket, isEqualToString
```

## A global hook the compiler calls on every throw

The Swift runtime exposes a global variable, `__swift_willThrow` — a function pointer. The compiler emits, at every throw site, «if this pointer isn't null, call it.» In a normal binary the pointer is null, so a throw pays nothing. Confirmed: in a plain executable the slot reads `0x0`.

Test frameworks install themselves there. Disassembling how Swift Testing registers its handler shows the mechanism — an atomic swap into that slot, keeping the previous handler so observers can chain:

```
_swt_setWillThrowHandler:
    ldr    x8, [GOT: __swift_willThrow]    ; address of the slot
    swpal x0, x0, [x8]                    ; atomic swap: install, return the old
    ret
```

So `__swift_willThrow` is not a function — it is a writable pointer that whoever observes throws swaps their handler into. A symbol scan pins down who installs what:

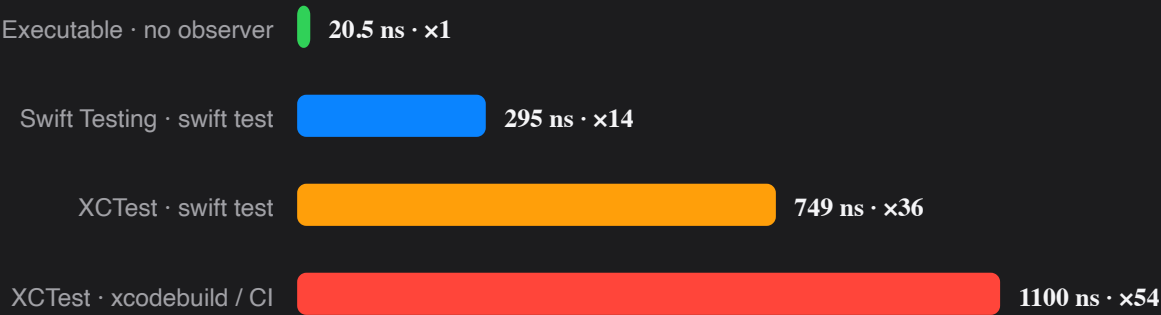
| BINARY                      | WILLTHROW REFS | WHAT IT MEANS                                  |
|-----------------------------|----------------|------------------------------------------------|
| XCTest.framework            | 0              | the framework itself is clean                  |
| libXCTestSwiftSupport.dylib | 2 + observer   | the hook lives here (XCTSwiftErrorObservation) |
| Testing.framework           | 4              | Swift Testing installs its own, too            |

That last row came from the community: u/ThatGuy739 reproduced the slowdown independently, located the hook in the support dylib, and flagged that Swift Testing references `__swift_willThrow` too.

## The same loop, three places

One function that throws on half its inputs, and the exact same loop compiled into an executable, an XCTest case, and a Swift Testing test. Only the host differs. Release build; time is per loop iteration (half of which throw), so the per-throw cost is roughly double.

ns per iteration · bare enum error · MacBook Pro M5 · Swift 6.3.3 · release



The cost is a fixed ~700 ns per iteration (~1.4 μs per actual throw). Linear from 1M to 8M throws — the hour-long stall was linear-but-brutal, not a runaway.

## It also leaks memory — this is the real defect

Speed is an annoyance. The memory is the part that is genuinely wrong. Each observed throw bridges to an autoreleased NSError, and XCTest only drains the autorelease pool at the end of the test method. Inside one long-running test, nothing is freed:

| 2,000,000 THROWS IN ONE TEST | MEMORY Δ  | PER THROW |
|------------------------------|-----------|-----------|
| Identical errors             | +4,031 MB | 2,113 B   |
| Distinct errors              | +4,031 MB | 2,113 B   |
| Wrapped in autoreleasepool   | +0 MB     | 0 B       |

### THE ONE TRUE DEFECT

Two million throws cost 4 GB, whether the errors are identical or all different. At fuzzing scale this is not slow, it is an out-of-memory crash. Draining the pool per iteration keeps it perfectly flat — which both proves the mechanism and hands you a fix.

## Twenty-one findings

Everything measured, grouped by what it does to you. **red = breaks things** · **amber = amplifies the cost** · **green = ruled out / good news**. Per iteration, release, swift test, unless noted.

| #  | FINDING                                                                                                                                             | METRIC       |
|----|-----------------------------------------------------------------------------------------------------------------------------------------------------|--------------|
| 01 | <b>Unbounded memory growth</b><br>Autoreleased NSError per throw, drained only at method end → OOM at scale.                                        | 2.1 KB/throw |
| 02 | <b>The disarm workaround poisons the process</b><br>Nulling the slot without restoring it silently disables throw observation for every later test. | footgun      |
| 03 | <b>The base per-throw cost</b><br>Fixed overhead the observer adds to every throw, independent of build config.                                     | ~1.4 μs      |
| 04 | <b>xcodebuild / CI stacks a second observer</b><br>Xcode loads Swift Testing alongside XCTest; the handlers chain. Same test, +51%.                 | ×1.5         |
| 05 | <b>Cost grows with call-stack depth</b><br>The observer captures the whole return-address stack. Throwing deep costs more.                          | up to ×3     |
| 06 | <b>Common APIs throw more than once internally</b><br>You pay per internal throw, for throws you never wrote (see §6).                              | ×2–3         |
| 07 | <b>Richer errors cost more</b><br>A CustomNSError with a userInfo dictionary bridges to a heavier NSError.                                          | ×1.4         |
| 08 | <b>Swift Testing is better, not a cure</b><br>2.5× cheaper than XCTest and no leak — but still 14× the executable.                                  | 295 ns       |

| #  | FINDING                                                                                                                                   | METRIC       |
|----|-------------------------------------------------------------------------------------------------------------------------------------------|--------------|
| 09 | <b>Typed throws fix the memory, not the speed</b><br>throws(E) leaks 0 B/throw but still costs ~730 ns under XCTest.                      | 0 B · 730 ns |
| 10 | <b>Off the test thread, the observer is nearly free</b><br>Its work only happens inside the test's block; background throws bail early.   | 34 ns        |
| 11 | <b>Async tests leak identically</b><br>async is not a mitigation; the pool still doesn't drain mid-loop.                                  | 2.1 KB/throw |
| 12 | <b>Typed throws are 4× faster — only in the executable</b><br>5.6 ns vs 20.5 ns with no observer; the observer erases the advantage.      | 5.6 ns       |
| 13 | <b>Not superlinear</b><br>Per-throw cost is flat from 1M to 8M throws. Brutal, but it doesn't accelerate.                                 | linear       |
| 14 | <b>Not MainActor hops</b><br>async 596 ns, actor 641 ns — the observer dominates; the hop adds ~45 ns.                                    | ruled out    |
| 15 | <b>No lock contention across threads</b><br>Background throwers scale linearly 1→8 threads.                                               | scales       |
| 16 | <b>Your error's description is not called per throw</b><br>0 calls over 1,000,000 throws; the string is built only for escaping failures. | 0 calls      |
| 17 | <b>Mixing XCTest and Swift Testing is safe</b><br>Each framework scopes its handler to its own tests; neither clobbers the other.         | no clobber   |
| 18 | <b>Disarming does not hide failures</b><br>XCTUnwrap and assertions record through a separate path; they still fail red.                  | safe-ish     |

Three more are the boundary conditions of the disarm fix (§7): it is process-global and non-atomic, relies on an internal symbol that may change between toolchains, and would meet pointer authentication if installing rather than clearing. Two remain conditional: an `os_log` per throw only if error-level logging is enabled, and Instruments signposts only with Instruments attached.

## §6 · THE MULTIPLIER

# You pay for throws you never wrote

A single call into a common Foundation API can throw more than once internally. Each internal throw is observed. Counted by installing a counting handler and calling each API once:

| ONE CALL TO...                                                      | INTERNAL THROWS |
|---------------------------------------------------------------------|-----------------|
| a hand-written throw (control)                                      | 1               |
| <code>JSONDecoder.decode</code> on malformed input                  | 2               |
| <code>FileManager.contentsOfDirectory</code> (missing)              | 2               |
| <code>FileManager.attributesOfItem</code> (missing)                 | 3               |
| <code>NSRegularExpression</code> / <code>Regex</code> , bad pattern | 1               |
| <code>Int("abc")</code> , <code>URL(string:)</code> — return nil    | 0               |

So a test that decodes malformed JSON pays roughly double; one that stats missing files, triple — for throws inside the standard library. Measured end to end, JSONDecoder on bad input runs 565 ns in an executable and 3,467 ns under XCTest — 6×.

x2

JSON decode (2 internal throws)

x3

stat missing file

~12 μs

worst-case per call

THE AMPLIFIERS COMPOUND

1.4 μs base × 3 stack depth × 2 API × 1.5 CI = ~12 μs per call — worst case: a deep throw, in a decoder, under CI. This is why the problem is not only a fuzzer's problem: any test that exercises error paths at volume pays it.

§7 · FIXES

# What actually gets you out

| APPROACH                               | SPEED     | MEMORY | CAVEAT                                         |
|----------------------------------------|-----------|--------|------------------------------------------------|
| Move the hot loop to an executable     | full      | full   | best for fuzzing/stress. No observer at all.   |
| Run the loop on a background thread    | full      | full   | throws there aren't attributed to the test.    |
| Wrap each iteration in autoreleasepool | slow      | full   | stops the OOM even if you stay in XCTest.      |
| Use typed throws throws(E)             | unchanged | full   | removes the leak; CPU cost stays.              |
| Return nil instead of throwing         | full      | full   | changes the API; right when «error» is common. |
| Disarm the observer around the loop    | full      | full   | sharp. Must restore, or poison later tests.    |

```
// Clears the willThrow observer for the duration of `body`, then restores it.
func withThrowObserverDisabled<R>(_ body: () -> R) -> R {
    let slot = dlsym(dlopen(nil, RTLD_NOW), "_swift_willThrow")?
        .assumingMemoryBound(to: UnsafeRawPointer?.self)
    let saved = slot?.pointee
    slot?.pointee = nil
    defer { slot?.pointee = saved } // or every later test loses observation
    return body()
}
```

If you can't leave XCTest, the honest minimum is one line: wrap the loop body in autoreleasepool { }. If you're fuzzing, don't fight it — put the loop in an executable.

## How serious is it, really

Calibrated, without the hype: a narrow, **test-time-only** sharp edge. It costs nothing in production — the hook is never installed in a shipped binary. It never affects correctness. It only hurts code that throws at very high volume inside a test — a small population, but a real one, and one that gets no warning.

0x

cost in production

1

genuine defect (the memory leak)

test-only

the whole footprint

### THE BOTTOM LINE

Of everything here, one item is a genuine defect rather than an expensive-by-design trade-off: the unbounded memory growth, which turns a slow test into an out-of-memory crash and is fixable by draining the pool. The rest is the runtime doing something useful without a documented cost or a cap. If you never throw in a tight loop under test, none of this will ever touch you.

## Sixteen lines

A function that throws on half its inputs, and the identical loop in two hosts. Run both; watch the executable finish in well under a second and the test crawl.

```
// Lib.swift
public enum ParseError: Error { case malformed }
@inline(never) public func parse(_ x: Int) throws -> Int {
    if x & 1 == 0 { throw ParseError.malformed }
    return x
}

// the loop – identical in the executable and in the XCTest case:
for i in 0..<20_000_000 { _ = try? parse(i) }
```

### The same loop, two hosts · what you should see

Executable  0.42 s

XCTest  16.80 s

The executable finishes in well under a second; the identical loop under XCTest crawls. Same work, only the host differs.

#### WHAT TO WATCH

Run both targets. If the test crawls while the executable flies, you are paying the willThrow tax. Watch a single core pinned at 100% and memory climbing under the test — that is the observer bridging every throw to an NSError that is only freed when the test method ends.

Check the figures on your own machine; that is the whole point.

§10 · CREDITS

## Who found what

| PERSON                                    | CONTRIBUTION                                                                                                                                               |
|-------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Yassin Daoud</b><br>github/MagicYassin | Found it while fuzzing Kalego; ran the full investigation and the twenty-one experiments.                                                                  |
| <b>u/ThatGuy739</b><br>r/swift            | Reproduced it independently on an M5, located the hook inside libXCTestSwiftSupport.dylib, and flagged that Swift Testing references _swift_willThrow too. |
| <b>u/Dry_Hotel1100</b><br>r/swift         | Pushed the hardest questions — is elapsed time a valid criterion, could this be MainActor hops? — which forced the rule-out experiments in §5.             |

MacBook Pro M5 · Swift 6.3.3 · macOS 26 · Xcode 26.6 · release unless noted. Measure it yourself before you believe any of it.